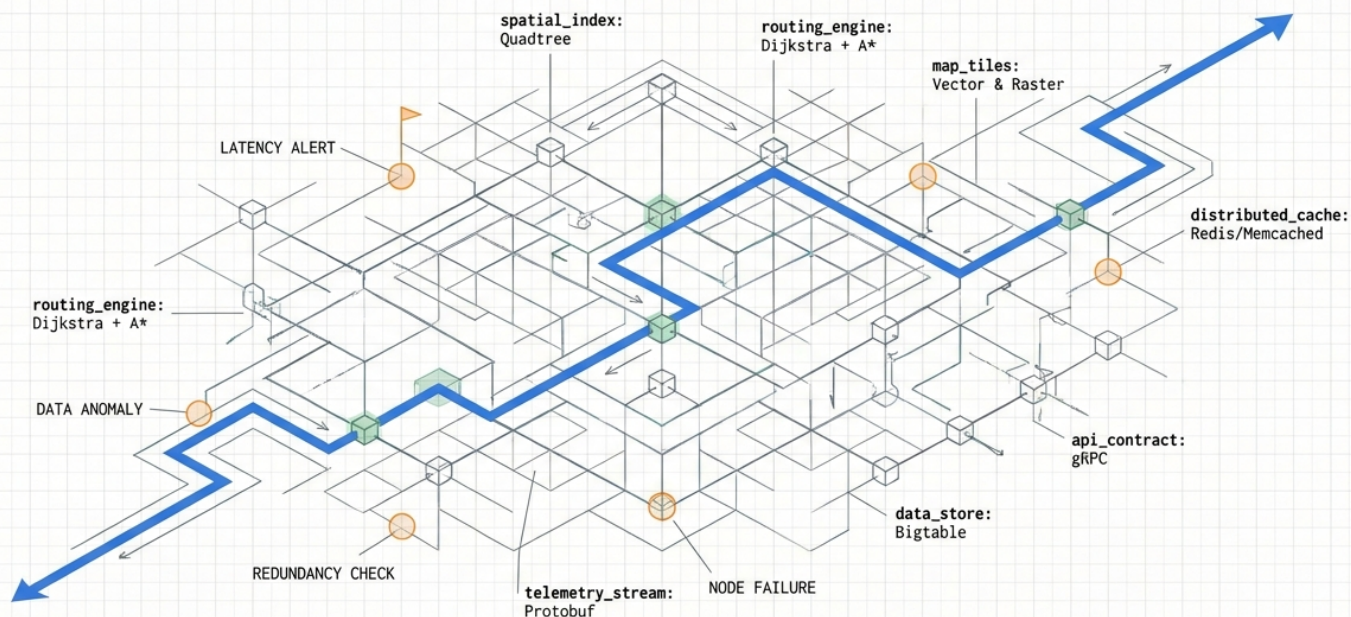


System Architecture: Google Maps

A structural blueprint for spatial search, distributed routing, and real-time telemetry.



System Spec Sheet: The Core Requirements

Functional Requirements



Location Identification

Pinpoint precise user latitude and longitude.

Route Recommendation

Calculate optimal paths by distance, time, and mode.

Navigation Directions

Provide step-by-step turn guidance.

Non-Functional Requirements

Availability

Mission-critical uptime for continuous navigation.

Scalability

Support massive global throughput and enterprise fleets.



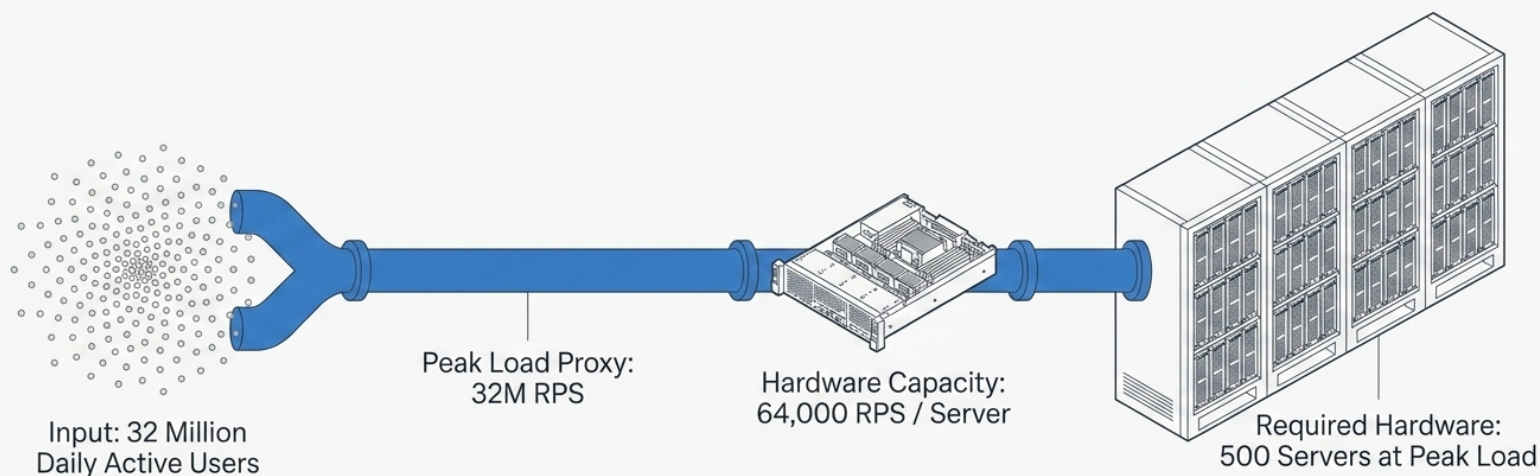
Latency Constraint

Route and ETA calculation strictly within 2-3 seconds.

Dynamic Accuracy

Predicted ETA must instantly reflect live actual travel time.

Resource Estimation: Peak Load Capacity



Storage Note

Static map requirements sit at ~20 PB of pre-processed road network data. Daily updates are negligible compared to this baseline.

Bandwidth Constraints: The Asymmetrical Payload

Incoming Requests

18,518 Requests per Second
Payload: 200 Bytes (Source/Dest)
Total Inbound: ~29.6 Mb/s

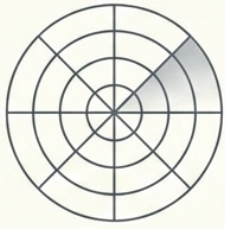
LOAD
BALANCER

Outgoing Responses

18,518 Requests per Second
Payload: ~2 MB (Visual Map + 5 KB Text)
Total Outbound: ~297 Gb/s

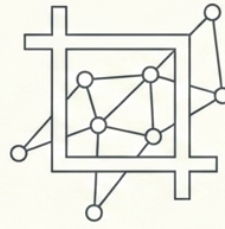
Architectural Takeaway: Outbound payload sizes dictate a massive CDN and edge-caching architecture to prevent network bottlenecking.

Macro-Architecture Components



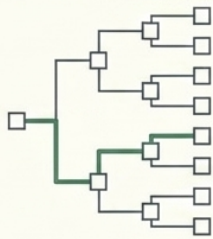
Location & Route Finders

Role: Ingest raw GPS/Cellular data and coordinate pathfinding requests.



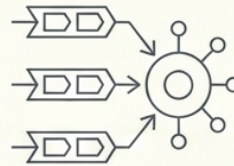
Area Search Service

Role: Calculates map bounding boxes and filters the global graph down to a local subgraph.



Graph Processing Service

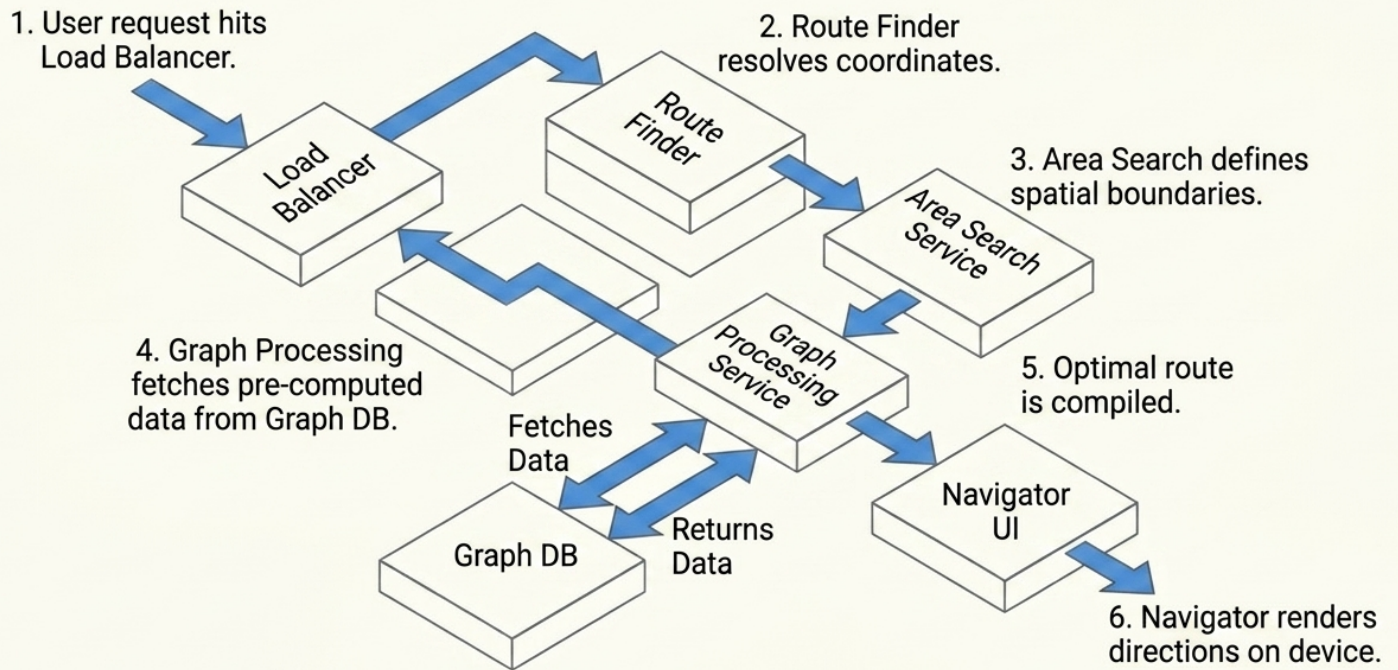
Role: Executes shortest-path algorithms purely on the filtered spatial subgraphs.



Distributed Pub-Sub (Kafka)

Role: Asynchronous event bus. Ingests live location streams for traffic analysis and rerouting.

The Request Pipeline: Happy Path



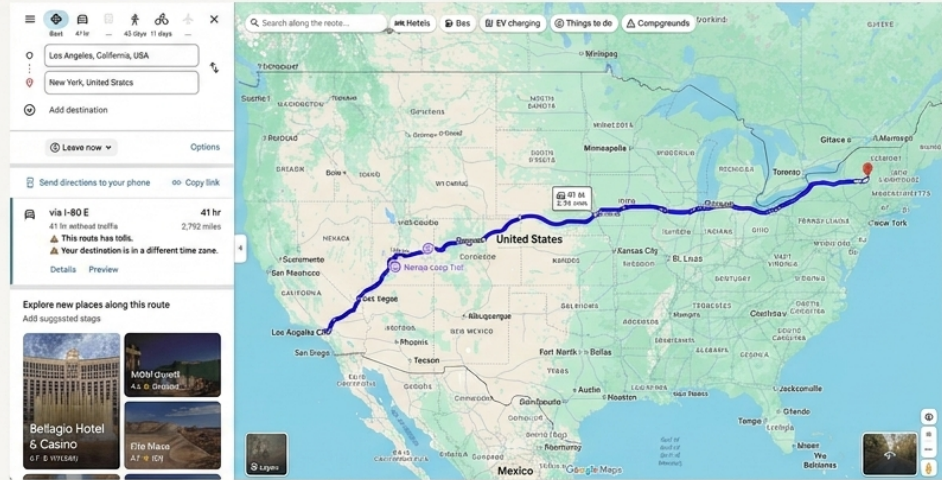
API Contracts & Client Output

Endpoint Definitions

```
show_user_location(location: boolean)  
// Establishes persistent streaming connection
```

```
find_optimal_route(source: string,  
destination: string, transport_type: enum)  
// Returns comprehensive Route Object payload
```

```
get_directions(current_location: LatLong)  
// Returns incremental text/audio step alerts
```



Deep Dive 1: Spatial Scalability

The Bottleneck

A global road network contains billions of vertices and edges.

The Math

Naive shortest-path algorithms (like Dijkstra's or A*) require $O(V \log V)$ compute. Running this dynamically against a global graph is mathematically intractable.

The Result

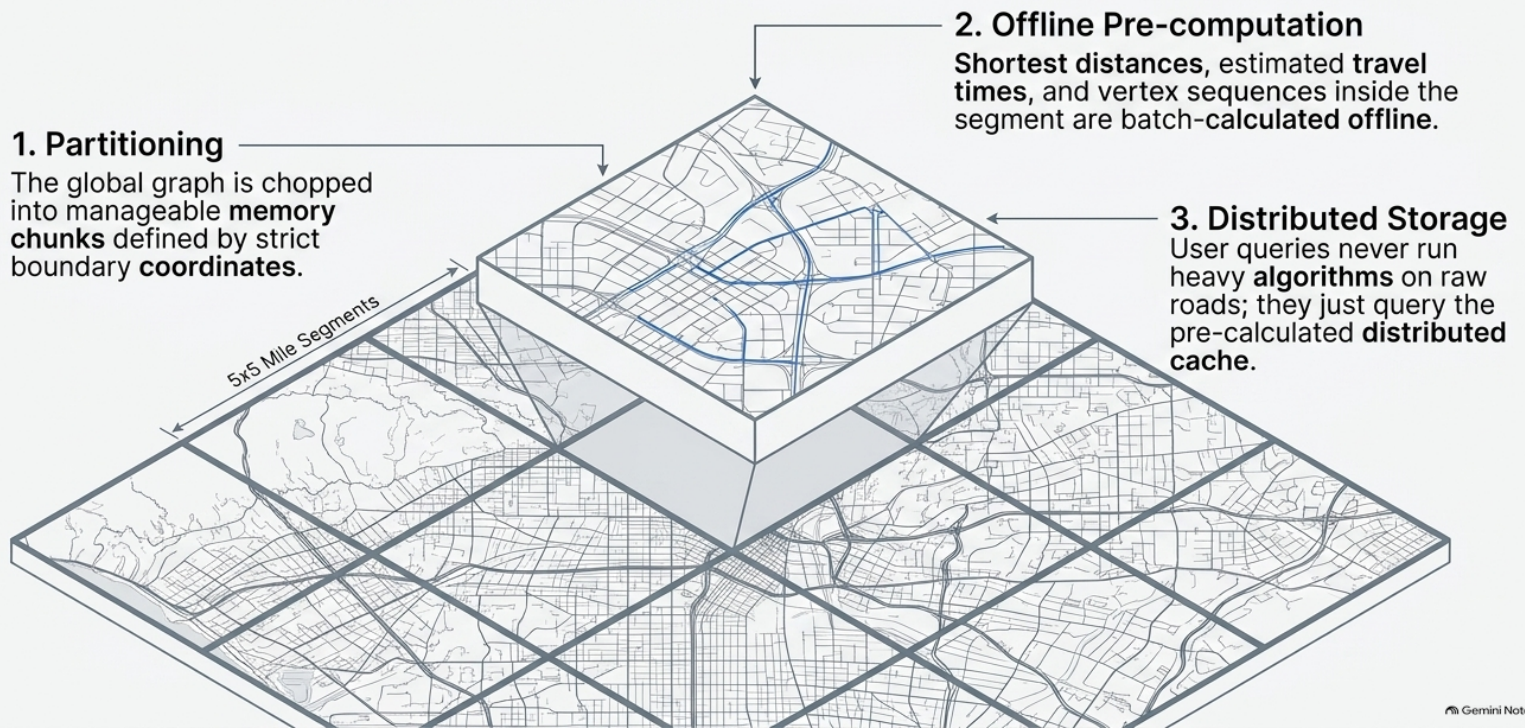
Serving millions of concurrent global queries would crush the 2-3 second latency constraint. Pre-computation and strict spatial limitation are required.



Diagnostic: Routing Strategy Comparison

Dimension	Naive Dijkstra (Global Graph)	Segment Routing (Pre-computed)
Query Scope	Traverses the entire contiguous global graph.	Traverses only filtered subgraphs and boundary exit points.
Compute Time	$O(V \log V)$ executed at runtime on millions of nodes.	$O(1)$ lookup for internal segments + local search on boundaries.
Cacheability	Impossible (infinite source-to-destination permutations).	Highly cacheable (internal segment paths are processed offline).

The Solution: Graph Segmentation



Micro-Architecture: Edge-Snapping



1. Identify Edge

Map the user's raw spatial coordinates to the nearest valid road segment.

2. Calculate Deltas

Measure the exact meter distance from the user to both terminating intersections (vertices).

3. Select Anchor

Add this micro-distance to the pre-computed routing table to lock the user onto the main navigable graph.

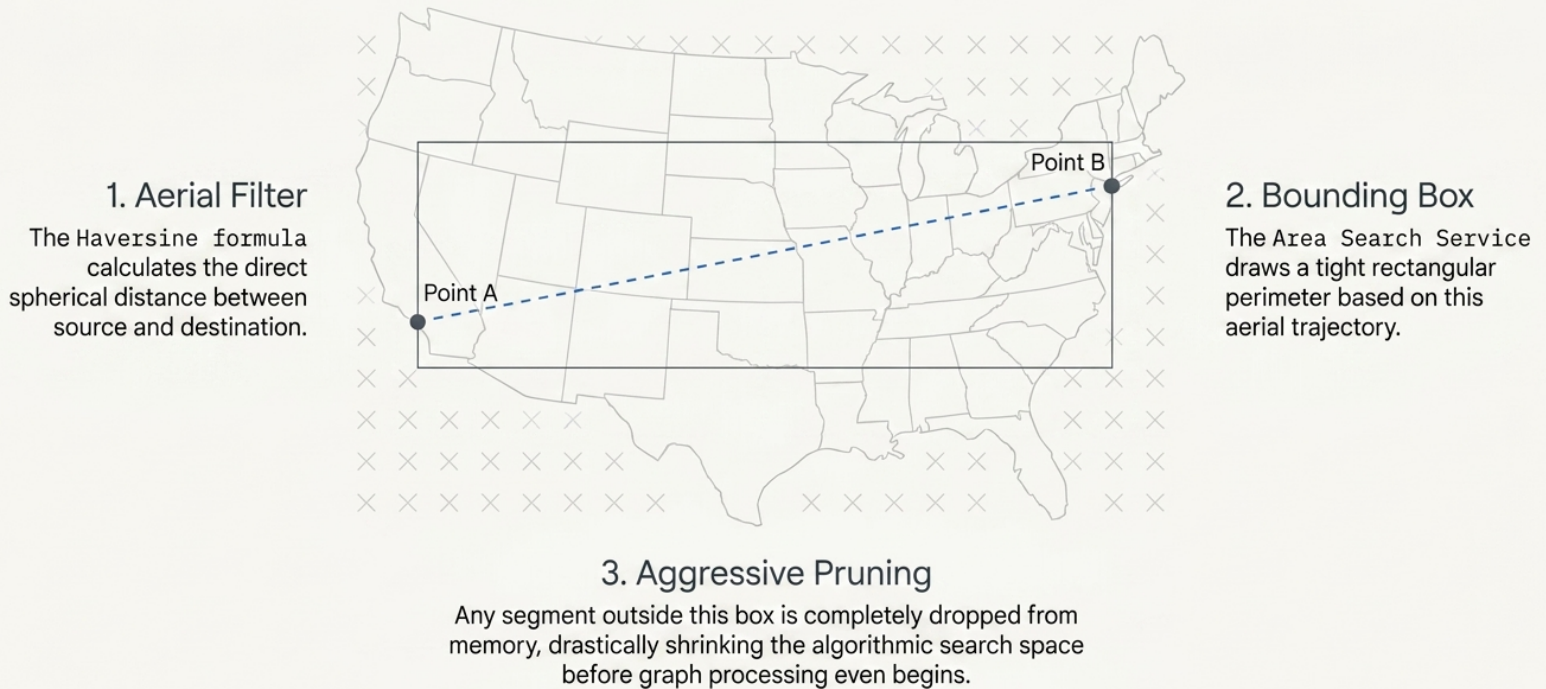
Macro-Routing: The Segment Hop



Exit Points as Bridges

When crossing multiple segments, the algorithm entirely ignores the internal roads of intermediate grids. It constructs a simplified high-level graph composed only of pre-calculated paths between boundary exit points.

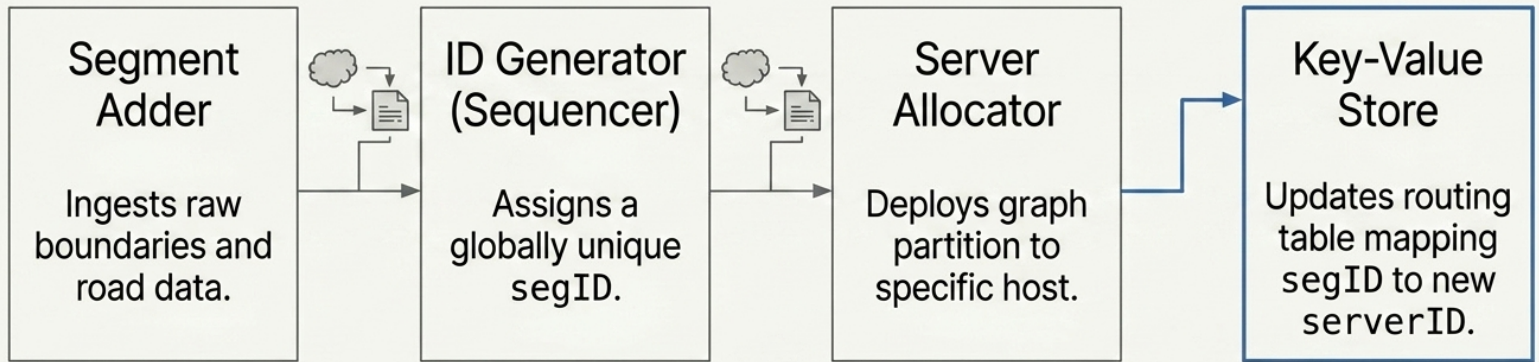
Search Space Pruning: The Haversine Bounding Box



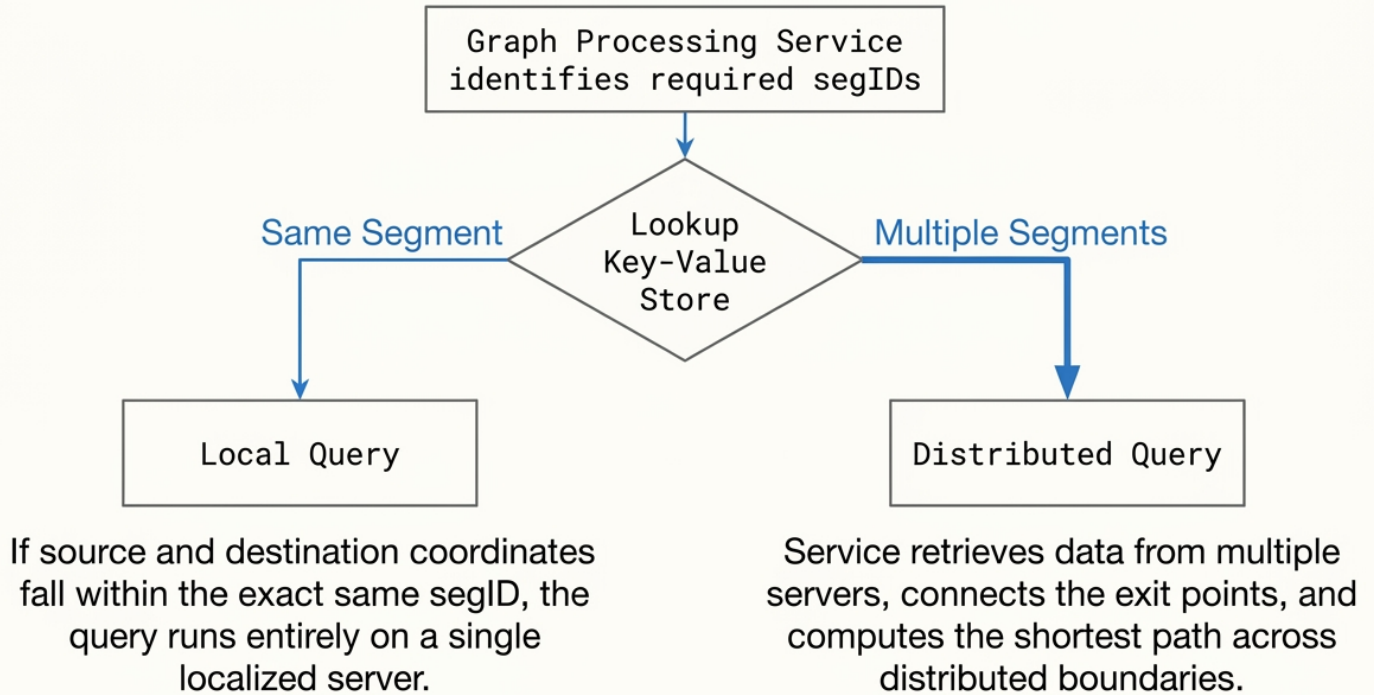
Polyglot Persistence: The Storage Schema

Key-Value Store	Graph Database	Relational DB
Purpose: Routing Metadata	Purpose: Topology	Purpose: Analytics
Schema Definition: segID -> serverID segID -> coordinates	Schema Definition: segID -> roadNetwork GRAPH segID -> neighbors List[]	Schema Definition: segID + edgeID -> hourRange segID + edgeID -> rushB00L
Use Case: O(1) lookup to instantly find which partition/server holds the segment data.	Use Case: Optimized exclusively for rapid traversal of nodes and edges within a segment.	Use Case: Historical time-series data for rush hour patterns and baseline edge weights.

Infrastructure Workflow: Segment Ingestion

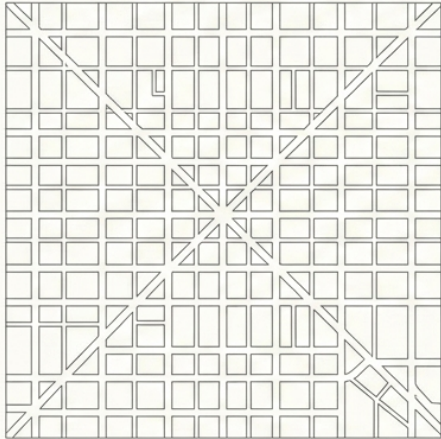


Query Execution: The Routing Split



Deep Dive 2: Temporal Accuracy

The Static Map (Scalability)



Pre-computation solves scale, but relies on fixed graph weights based purely on physical distance.

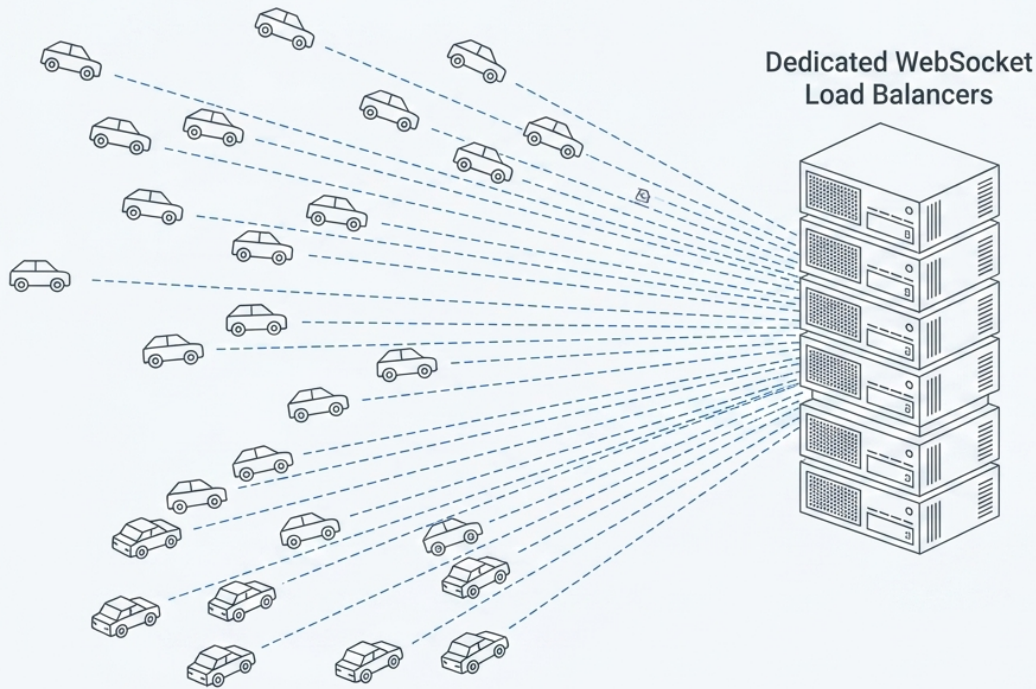
The Living Organism (Accuracy)



Travel time is dictated by dynamic variables: traffic density, accidents, and closures. The system must perpetually ingest live data to update edge weights.

A static map is useless if it cannot see a crash that happened 5 minutes ago.

Telemetry Ingestion: Persistent Connections



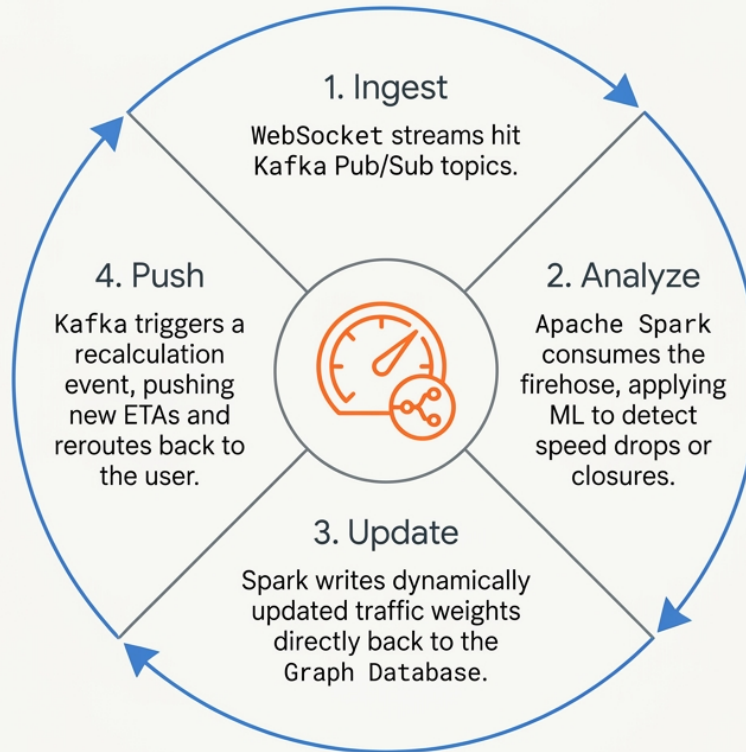
Why WebSockets?

GPS, Wi-Fi, and cell tower data must stream continuously from moving fleets. Polling via standard HTTP creates unacceptable request overhead. WebSockets enable low-latency, two-way, persistent communication between the navigator device and edge servers.

Connection Management

Dedicated load balancers handle the sheer volume of persistent TCP connections, aggregating the stream of (Device ID, Timestamp, Location) tuples into the backend.

The Telemetry Feedback Loop

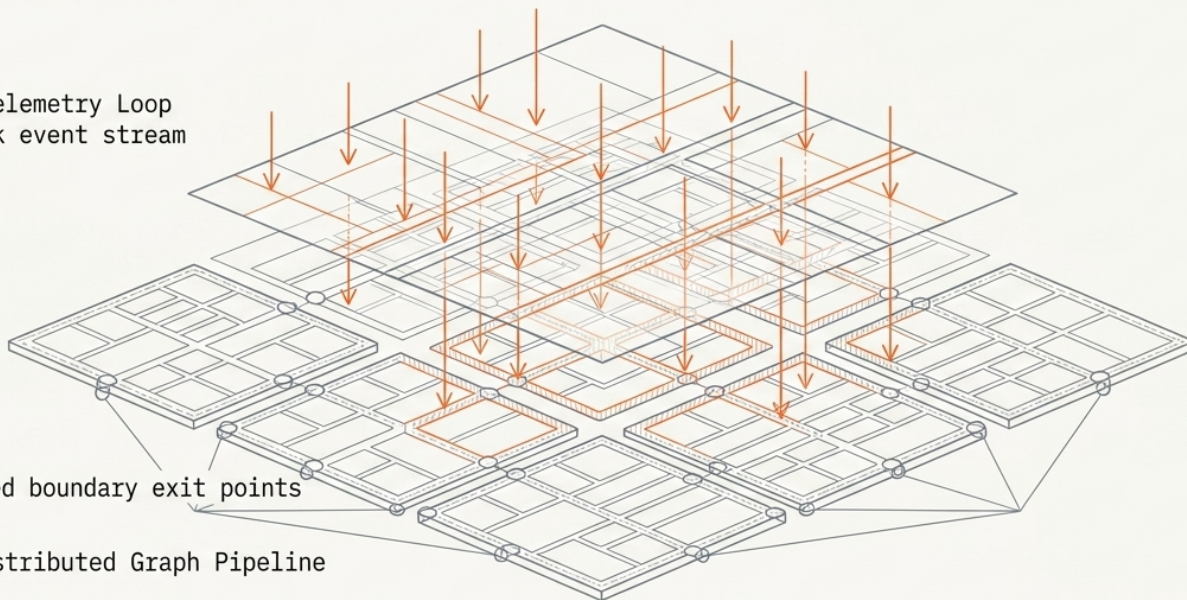


Synthesis: The Master Blueprint

Dynamic: Telemetry Loop
Kafka/Spark event stream

Pre-computed boundary exit points

Static: Distributed Graph Pipeline



Architectural Synthesis

The Distributed KV/Graph pipeline acts as the skeleton—structural and infinitely scalable via segmentation.
The WebSocket/Kafka loop acts as the nervous system—adaptive and real-time. Neither functions in isolation.

Evaluation: System Constraints Met

	Scalability	Achieved via Graph Segmentation and Bounding Box pruning, ensuring $O(V \log V)$ never runs on the global graph.
	Low Latency (2-3s)	Achieved via batch offline pre-computation of internal segment paths and $O(1)$ Key-Value routing lookups.
	Dynamic Accuracy	Achieved via the persistent WebSocket + Apache Spark ML pipeline, continuously mutating graph edge weights based on live fleet telemetry.
	High Availability	Achieved via distributed data replication partitioning across KV stores, Graph DBs, and Kafka broker clusters.